Contents

A reader's guide to the empirical and pricing code **2**
PDF editions . . . 2
How the components connect . . . 2
Three meanings of a sample . . . 3
Reading existing documentation . . . 3

Data construction and descriptive statistics **4**
Contents . . . 4
1. Know the unit of every table . . . 4
2. Conversion and concatenation . . . 5
3. Merging the two source families . . . 5
4. Calendar matching and `master_dat.py` . . . 6
5. `summ_stats.py`: projection and sample definitions . . . 7
6. Household summaries and demographics . . . 7
7. Flavor recovery and two taxonomies . . . 8
8. Descriptive choice dynamics . . . 8
9. Why the summary code writes temporary partitions . . . 10
10. Running, outputs, and verification . . . 10

Demand estimation: from purchase histories to a simulated likelihood **12**
Contents . . . 12
1. What is being estimated? . . . 12
2. Source map and control flow . . . 12
3. Turning purchase lines into weekly choice observations . . . 13
4. The history index, derived and worked through . . . 14
5. Utility and the four specifications . . . 15
6. The likelihood: why integration comes after summing the history . . . 16
7. The analytic gradient and numerical optimization . . . 16
8. The price control, step by step . . . 17
9. Household bootstrap and its scope . . . 18
10. Running and reading the results . . . 18

Dynamic pricing: a two-period symmetric-switching model **20**
Contents . . . 20
1. The economic problem . . . 20
2. Architecture and source of truth . . . 21
3. Utilities: why the history coefficient is signed . . . 21
4. Solving the terminal period . . . 22
5. First-period profit and the effective-cost interpretation . . . 22
6. The numerical root solver and why its solution is unique . . . 23
7. General comparative statics: the implicit-function calculation . . . 24
8. Gamma comparative statics and their interpretation . . . 24
9. Three price ratios that answer different questions . . . 25
10. Using the API and reproducing outputs . . . 26

A reader's guide to the empirical and pricing code

These guides describe the source present on November 1, 2026, including current working-tree changes. Read them in pipeline order:

1. Data construction and descriptive statistics.
2. Demand estimation and inference.
3. Dynamic pricing and comparative statics.

Each chapter connects mathematical definitions to functions, intermediate tables, algorithms, outputs, examples, and interpretation. The guides document the code; they do not certify the private server data or the model assumptions.

PDF editions

There are two master books:

- The current model and pipeline guides: data construction, estimation, inference, pricing, and mathematical derivations.
- The Python 101 code walkthrough: Python foundations, worked reading lessons, actual source listings, each function's purpose and alternatives, and statement-by-statement syntax notes.

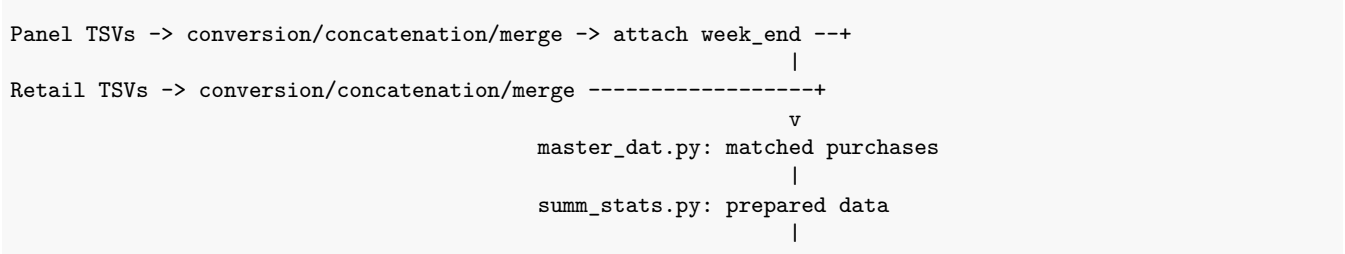
Each book covers all three code directories and includes a generated table of contents, PDF bookmarks, wrapped code examples, and page numbers. The Python walkthrough includes nested functions, test methods, top-level scripts, and empty package files; its coverage manifest checks for omitted function/statement entries.

Rebuild from the repository's `Code` directory:

```
python3 final_draft/build_code_guides.py
python3 final_draft/build_python101.py
```

These use local Pandoc and XeLaTeX and read no private data. The original guides remain in each code directory. Editable Python lessons are in `python101/`; the complete generated walkthrough source is `PYTHON101_WALKTHROUGH.md`. The first builder's optional `--individual` flag also recreates the earlier individual chapter PDFs; the default writes only the collected current-guide PDF.

How the components connect



```
                                        est.py: likelihood and inference

Fixed type + calibration -> supply backward induction -> derivatives/figures
```

The arrow from estimates to supply is deliberately absent: the current supply entry point does not load demand estimates. A researcher must explicitly define that bridge. Demand has three inside flavor groups and a cumulative state index; supply has one inside product, an outside choice, two periods, and signed last-choice states. The two gamma parameters therefore do not share a mechanical unit conversion.

Three meanings of a sample

The full panel consists of recorded purchases in `merged_panel.parquet`; it is not a roster of every enrolled household including households with no purchases. The matched yogurt sample contains purchases that survive retail matching, positive-price filtering, and the two-purchase-week restriction. The final estimation sample further requires compatible exposure-store menus and usable instruments, then reapplies the two-usable-week restriction. Its household count can be smaller than the descriptive sample's count.

Reading existing documentation

Use the Python source to settle runtime defaults. The estimation README contains older statements followed by amendments: current bootstrap replications default to **49**, simulation draws to **128**, and main starts to **3**. For supply, `MODEL_CONVENTIONS.md` and `comparative_statics/canonical.py` describe the current symmetric-switching model. Several other supply notes and retained output files describe superseded models. The chapters below identify those boundaries.

Data construction and descriptive statistics

This chapter explains the scripts in `cleaning_merging/`, the summary pipeline in `summ_stats.py`, and the flavor-recovery utility. It describes current source behavior, including consequential assumptions in older scripts. Continue with the estimation chapter after the prepared purchase table is built.

Contents

1. Data units and the overall pipeline
2. Conversion and concatenation
3. Panel and retail merges
4. Calendar matching and the yogurt sample
5. Summary input projection and sample restrictions
6. Household demographics and participation
7. Flavor recovery and classification
8. Shares, concentration, transitions, and spells
9. Memory management
10. Outputs, commands, and interpretation

1. Know the unit of every table

The data pipeline combines household purchase records with store-level retail prices. These sources answer different questions. Household records say who bought which UPC and how many packages. Retail records supply a store/week/UPC price. Joining them creates a purchase line with a matched price; it does not create a complete retail assortment panel.

Object	Intended unit	Important key
Purchases	A recorded UPC purchase line	Trip ID and UPC; multiple lines may exist
Trips	A shopping trip	`trip_code_uc`
Panelists	Household demographics in a panel year	Household, panel year
Panel products	UPC metadata	UPC
Retail movement	Store/UPC/week record	Store, UPC, week end
Retail stores	Annual store metadata	Store, year
Extra products	Annual UPC descriptors	UPC, year
Summary household table	One retained household	Household
Sparse flavor quantities	Household/week/flavor	Household, Saturday week, flavor
Estimation input	Original retained purchase line plus resolved label	Original fields retained

These are intended keys. Older merge scripts do not enforce every uniqueness condition with join validation. A

left join can multiply rows if the right side has duplicate keys. A successful run and a nonempty output therefore do not prove that the joins are correct.

The logical execution order is panel conversion → panel concatenation → panel merge → panel week attachment; independently, retail conversion → retail concatenation → retail merge; then master matching → summaries → estimation. Paths in the older scripts are hard-coded to `/scratch/dtm63837/Kilts_Panel`. The summary script exposes paths through command-line arguments.

2. Conversion and concatenation

2.1 `panel_convert.py`

This reads annual panelists, purchases, and trips and selected master files, converts TSV to Parquet, lowercases names, then creates narrower `cleaned_` files. TSV scanning uses tab separators, disables quote parsing, and infers types from a prefix. Some master reads use lossy UTF-8 decoding.

The purchase projection retains trip ID, UPC, quantity, total price paid, and deal flag. Trip projection supplies household, purchase date, store, and year. Panelists provide income, household size, head ages/employment, race, Hispanic origin, and DMA. Product metadata supplies module, group, brand, and size fields.

Projection reduces subsequent I/O, but a column dropped here cannot be recovered from the cleaned file. A new downstream analysis needing another raw variable must add it to the projection and rebuild the affected stages.

2.2 `retail_convert.py`

Annual sources include `products_extra`, `rms_versions`, stores, and movement modules 3603 and 3612. Master sources include products and brand variations. The current extraction loop covers 2006–2015. Movement projections keep store, UPC, week end, and price; they do not retain every original retail field.

The cleaner matches table names by longest prefix first, so `products_extra` does not accidentally receive the shorter `products` projection. It skips files already named `cleaned_`. Conversion loops report missing files; those messages must be checked because a loop can continue with incomplete annual coverage.

2.3 Concatenating years

`panel_concat.py` scans cleaned annual files into `combined/panelists.parquet`, `combined/purchases.parquet`, and `combined/trips.parquet`. `retail_concat.py` builds combined store, extra-product, module, and RMS-version tables. It explicitly casts common schema-conflict fields before `vertical_relaxed` concatenation: flavor/size codes become strings, UPC/store IDs Int64, and panel year Int32.

Concatenation stacks rows; it does not establish key uniqueness or remove duplicate years. Wildcard-selected files are part of the effective input. A stale annual file in the directory can silently become part of a rebuild. `panel_dat.py` and `retail_dat.py` are empty placeholders in this checkout.

3. Merging the two source families

3.1 `panel_merge.py`: purchases are the anchor

The merge chain is

```
purchases INNER JOIN trips ON trip_code_uc
          LEFT JOIN panelists ON household_code, panel_year
          LEFT JOIN products ON upc
    -> Data/merged_panel.parquet
```

Join keys are cast consistently before merging. The inner trip join can discard unmatched purchase lines; the demographic/product joins are intended to retain lines while adding missing metadata where no match exists.

The script reports row count, count relative to purchases, key nulls, file size, and columns. It raises on an empty result. However, 100% overall row retention can hide offsetting losses and duplication; verify right-key uniqueness

and unmatched keys when auditing a production rebuild. The printed claim that the file is ready is a status message, not a full statistical validation.

3.2 `retail_merge.py`: movements are the anchor

Retail movements from modules 3612 and 3603 are concatenated. Calendar year is derived from the integer week end using `week_end // 10000`. Then metadata is attached through left joins:

1. RMS versions, deduplicated to the first row per UPC, supply `upc_ver_uc`.
2. Products match UPC and version, deduplicated on that composite key.
3. Stores match store and calendar year, retaining the first duplicate.
4. Extra products match UPC and year, prioritizing nonnull flavor descriptions through a sort before retaining one row.
5. Brand variations match the first row per brand code.

Output is `Data/RMS/merged_rms.parquet`. Selecting a first row prevents many joins from multiplying records, but it is also a substantive reconciliation rule. For example, first-per-UPC RMS version selection discards version/year variation. The source does not establish that the chosen row is economically preferred.

Post-merge diagnostics compare movement and output row counts and report nonnull metadata rates. A nonnull brand-code field is not always evidence of a successful brand-dictionary match, because the code may inspect the original join key. Read each diagnostic as the particular column check it implements.

4. Calendar matching and `master_dat.py`

4.1 Checking the upstream Sunday convention

`panel_week.py` reads `merged_panel.parquet`, adds integer `week_end`, writes a temporary replacement, and replaces the original file with `os.replace`. Its implemented day offset is

$$6 - (\text{weekday}(d) \bmod 7).$$

With Polars Monday=1 through Sunday=7, Monday advances five days and Saturday zero. Sunday has remainder zero, giving an offset of six days. This differs from a direct `6-weekday` expression, which would send Sunday backward. The exact source formula must be evaluated with the installed Polars weekday convention before attributing lost Sunday rows.

The current summary and estimation formula is $(6 - \text{weekday}(d)) \bmod 7$; under the documented 1–7 convention, it agrees with the upstream formula for all weekdays. Older handoff warnings about Sunday loss should therefore be treated as an audit question about the code and data version actually used to build the input, rather than proof of a bug in the checked-in formula.

Week end is a matching key, so a historical erroneous build can still matter: later recomputation does not recover rows discarded by an earlier inner join. For a rebuild, explicitly check Sunday, Saturday, year-end, and leap-year dates against the retail calendar, and compare the saved `week_end` to recomputation.

4.2 Master matching

`master_dat.py` reads `Data/merged_rms.parquet`, while `retail_merge.py` writes `Data/RMS/merged_rms.parquet`. This is a real path mismatch in the current scripts. A prepared server file or manual copy may have bridged it; the source does not do so automatically. Resolve the intended path before a fresh run.

The master script projects retail columns to year/week/UPC/store, module code, and price; filters module codes 3603 and 3612; and keeps the first quote for each composite key. It collects this reduced retail table into memory, then processes panel purchases one year at a time.

For each year, it inner joins on

$$(\text{panel_year}, \text{week_end}, \text{upc}, \text{store_code_uc}).$$

Only purchases with matching retail keys survive. This selects on retail coverage, calendar alignment, UPC coding, and store coding. The result is not every household yogurt purchase. After yearly outputs are concatenated into `Data/yogurt_estimation_sample.parquet`, temporary year files are removed.

Because the match includes both panel year and calendar week end, year-boundary conventions require attention: a late-December purchase whose Saturday falls in January can have a panel year different from the retail-derived year. That possibility should be audited in the private data.

The source deduplicates retail keys by keeping the first quote rather than checking for contradictory prices. Estimation checks conflicts in its merged input, but cannot detect contradictions already discarded upstream.

The master audit reports entity counts, key nulls, years, price distribution, and file size. It does not report all purchase losses against the original yogurt universe or establish that all surviving prices represent the desired package/offer convention.

5. `summ_stats.py`: projection and sample definitions

`scan_purchases` starts from a lazy Parquet scan. It requires household, year, purchase date, trip, UPC, and quantity. It projects optional demographics into a common schema; absent columns become null strings. Income can originate as `household_income` or `income`; Hispanic origin can originate as `hispanic_origin` or `hispanic`.

Dates are parsed from their string prefix and quantities become Float64. Numeric-looking UPCs are canonicalized as strings, removing leading zeros and trailing `.0` without passing through floating point. Price uses `price`, then `price_rms`, or null if neither exists. This projection is deliberately narrower than the final estimator input: the later export reopens the original file to preserve store IDs and all other purchase columns.

`run` maintains two populations. `full_panel` uses `merged_panel.parquet` in the selected years and does not impose matched-price or repeated-yogurt rules. `matched_yogurt` starts from the master sample, removes missing/nonfinite/nonpositive prices, and requires two distinct Saturday-ending purchase weeks. The two-week rule is implemented by comparing a household's minimum and maximum week; multiple trips or lines in one week still count as one week.

The saved descriptive matched sample precedes estimator menu and IV exclusions. It can include a household later dropped for fewer than two usable likelihood weeks. Full-panel summaries cover households represented in purchase records, not every potential household in an enrollment roster.

6. Household summaries and demographics

`household_summary` produces one row per household, so prolific shoppers do not receive extra weight in demographic tables. Counts include distinct panel years, first/last year, distinct trips, shopping dates, distinct UPCs, purchase lines, and total packages. Trips use `trip_code_uc`; shopping days use purchase date. They need not agree because two trips can occur on the same day.

Demographic history is projected, deduplicated, and stably sorted by household, year, and date. Each field takes its earliest nonmissing value independently. Consequently, the resulting household row need not correspond to a single original date if fields became available at different times. Missing means null; unusual nonnull source codes need their own codebook interpretation.

Zero male/female head-age codes become missing. Combined `head_age` prefers female head age when available, then male head age. This is a category-selection rule, not an average of the heads' ages. Hispanic origin string `"1"` overrides the main race category to `"Hispanic"`; `race_original` preserves the prior value.

Approximate exposure is

$$W_i^{approx} = Y_i^{observed} \times 365.25/7, \qquad \text{trip rate}_i = T_i / W_i^{approx}.$$

A household observed in 2006 and 2008 has two observed years, not three. This avoids filling a missing year, but assumes full-year exposure in each observed year. It is not verified enrollment time. In the matched sample, trips count trips represented by retained yogurt purchases, not all shopping.

`demographic_tables` reports categories including an explicit `Missing` group, using all households in the denominator. Tables are unweighted. Label dictionaries for race, income, Hispanic origin, and age are initially empty; populate them only with verified codebook mappings. Numeric income/head-age means in the overview are means of source category codes, not dollars or years.

`save_table` prints each table and writes paired CSV and LaTeX files. The LaTeX export escapes source text and uses two decimal places. Year coverage is computed one year at a time from purchases, retaining exact distinct trip and household counts rather than summing household averages.

7. Flavor recovery and two taxonomies

7.1 Supplementing the reference

`recover_flavor_reference` normalizes UPCs and anti-joins annual product metadata against existing reference UPCs. Only UPCs absent from the reference are added; it does not overwrite a bad or blank existing descriptor. Candidate additions must have a nonmissing usable flavor description. When annual years exist, the earliest candidate metadata is selected per UPC; without years, the first candidate is kept.

The supplementary rows are saved separately beside the combined recovered reference. The function refuses to overwrite the original reference. `run` uses this automatically if the annual product source exists; otherwise it logs that only the supplied CSV will be used. `recover_flavors.py` exposes recovery as a standalone command with `--flavors`, `--products`, and `--output`.

`valid_flavor_labels` is a small descriptor-validity helper. The active recovery code also expresses validity filters directly in its lazy query; the helper's presence should not be read as an additional transformation applied everywhere.

7.2 Detailed versus grouped labels

`flavor_reference(..., detailed=True)` lowercases and trims `flavor_descr`, collapses whitespace, maps blank descriptions to `unknown`, and maps `unflavored` to `plain`. These detailed labels drive the main summary directory.

The grouped pass searches combined flavor and optional product descriptions: plain/unflavored first, then berry/straw/blue/rasp/black/cran/cherry/wildberry, and otherwise other. This broad text rule is reproducible, but it is not a hand-verified flavor ontology. "Cherry" enters berry by construction, and a description matching both plain and berry enters plain.

Duplicates are reconciled by the first row per normalized UPC. Conflicting classifications are counted and optionally exported. Missing descriptors on a known UPC still classify: `unknown` in detailed labels, usually `other` in groups. An entirely unmatched purchased UPC causes `yogurt_weeks` to export an unmatched UPC audit and raise, rather than silently deleting the purchase.

The grouped pass writes `estimation_flavor_reference.csv` and embeds its labels in the final prepared purchase file. This avoids reclassification during the normal estimation path.

8. Descriptive choice dynamics

8.1 Sparse weekly quantities

`yogurt_weeks` joins the reference and aggregates quantity by household, Saturday week, and flavor. Only observed positive quantities occupy rows. It audits unmatched labels, invalid quantities, and bad prices. Nonpositive/nonfinite quantities stop processing; the summary stage does not enforce integer quantity as strictly as the estimator does.

Keeping the table long avoids creating a dense household-week-by-detailed-flavor matrix. No unobserved weeks or zero-purchase rows are inserted.

8.2 Aggregate shares and household concentration

The aggregate package share is

$$\text{Share}_j = 100\frac{\sum_{it} q_{ijt}}{\sum_{itk} q_{ikt}}.$$

It weights heavy purchasers more than light purchasers. The saved share table omits flavors rounding to zero at two displayed decimals while retaining the original denominator; displayed shares can therefore sum slightly below 100.

For household totals $Q_{ij} = \sum_t q_{ijt}$, define

$$s_{ij} = Q_{ij} / \sum_k Q_{ik}, \qquad HHI_i = \sum_j s_{ij}^2, \qquad L_i = \max_j s_{ij}.$$

`yogurt_household_flavors.csv` stores number of flavors bought, HHI, and largest share. With counts 6,3,1, HHI is .36+.09+.01=.46 and largest share .6. With two equally purchased flavors HHI=.5; with one flavor HHI=1. It is on a 0–1 scale.

Detailed and grouped HHI answer different questions. Combining categories adds cross terms: merging a and b raises HHI by $2s_a s_b$. Thus grouped HHI is at least detailed HHI for the same purchases. The mean household HHI weights each household equally, unlike the aggregate package shares.

`make_hhi_figures` uses all household values, with 40 equal-width bins from zero to one for the histogram and an empirical step CDF. Histogram weights sum to 100%; these are percentages, not density units. The CDF gives the share of households with HHI at most a threshold. Neither is truncated to 20 flavors.

8.3 Transition probabilities

`flavor_summaries` builds one basket per household-week. A basket has a `single_flavor` label only when it contains exactly one flavor. A pair is eligible only when current and immediately preceding observed baskets are both single-flavor and exactly seven days apart. Mixed baskets and gaps break pairs.

For transition counts n_{ab}, rows are previous flavor a, columns current flavor b, and

$$\widehat{P}(b \mid a, \text{eligible}) = n_{ab} / \sum_k n_{ak}.$$

Stays are included in the denominator. A row with no eligible transitions has undefined probabilities, represented as missing. The switch rate is the percentage of eligible pairs with different flavors; it is not the share of all households that ever switch.

For example, berry on Jan 2, berry on Jan 9, plain on Jan 16, and berry on Jan 30 produces berry→berry and berry→plain. Jan 30 contributes no transition because Jan 23 is missing. Berry's observed transition probabilities are .5 stay and .5 plain. This describes adjacent single-flavor purchase behavior, not all consumption transitions.

8.4 Spells and graphs

A spell continues only across eligible same-flavor adjacent weeks. Changing flavor, encountering a mixed basket, crossing a gap, or starting another household creates a new spell identifier. Mixed baskets are excluded from single-flavor spell output. Spell length counts consecutive observed calendar weeks, not packages, days, or elapsed time between arbitrary purchases.

The mean spell length averages spells; a household with many spells contributes many observations. `yogurt_dynamics` also reports mixed-basket frequency, weeks above seven packages, eligible pairs, and the median observed week gap. The above-seven measure diagnoses the optional estimator capacity convention; the summary code does not discard these baskets.

Flavor-share and transition figures show up to the 20 most purchased flavors. Transition probabilities are calculated before subsetting, so displayed rows need not sum to one if destinations lie outside the plotted subset. Saved transition tables retain the fuller taxonomy.

9. Why the summary code writes temporary partitions

Lazy scans defer work; they do not guarantee constant memory. Exact distinct counts and group aggregations retain state, and repeated scans of a giant purchase source are expensive. `partition_purchases` hashes household IDs with seed zero into `hash(id) % batches`, then writes physical Parquet partitions. All rows for a household remain together, preserving history and exact distinct counts across years.

`household_batches` reads one partition at a time, builds household summaries, and writes small intermediate files. Those same purchase partitions are reused for detailed and grouped weekly flavor aggregation. Household demographic history is narrowed and deduplicated before sorting once across all fields; exact purchase totals are computed separately from all lines.

This avoids rescanning the original source once for every household batch and avoids a dense detailed-flavor pivot. It still needs temporary disk space and can retain large aggregation state within a partition. Increasing `--household-batches` reduces typical per-partition size, with more files and overhead. Default is 16. Hash partitioning is not sampling and should not change the reported statistics.

The implementation requires Polars >=2.0 and checks for `pl.PartitionBy`. Temporary summary work lives under the output folder and is cleaned up when the temporary-directory context exits. Basket/spell checkpoints are also temporary. Persisted prepared data, CSVs, LaTeX tables, and graphs remain.

10. Running, outputs, and verification

Legacy cleaning scripts execute at module top level and can write or replace large server datasets. Read their path constants and file dependencies before running them; importing them is not a harmless way to inspect functions. `summ_stats.py` and `recover_flavors.py` have guarded entry points.

From `final_draft/data_code`:

```
python3 summ_stats.py --data-dir /path/to/Data \
  --flavors /path/to/Latest_Flavor_2010.csv \
  --products-extra /path/to/Data/RMS/combined/products_extra.parquet \
  --output /path/to/Data/summary_outputs \
  --first-year 2006 --last-year 2015 --household-batches 16
```

`--no-figures` skips plotting while retaining summaries. To recreate HHI figures from saved household summaries alone:

```
python3 summ_stats.py --hhi-only --output /path/to/Data/summary_outputs
```

This requires the detailed `yogurt_household_flavors.csv` and also updates grouped HHI graphs when that grouped file exists. It does not reread purchases.

Output	Meaning
`full_panel_households.csv`	Household summaries from full purchase panel
`matched_yogurt_households.csv`	Household summaries after matched-sample price/two-week filtering
`{prefix}_{demographic}.csv/.tex`	Unweighted categorical distributions with missing group
`{prefix}_overview.csv/.tex`	Household counts, means, rates, and race percentages
`{prefix}_year_coverage.csv/.tex`	Exact annual sample coverage
`yogurt_price_coverage.csv/.tex`	Lines excluded by the matched-price condition
`recovered_flavor_reference.csv` and companion rows	Original-plus-supplemented descriptor reference
`detailed_flavor_reference.csv`	Resolved detailed UPC labels
`estimation_flavor_reference.csv`	Resolved three-group UPC labels
`yogurt_sample_audit.csv/.tex`	Detailed-pass purchase/label/quantity audit
`yogurt_flavor_shares.csv/.tex`	Package-weighted flavor shares
`yogurt_household_flavors.csv`	Household concentration and breadth

Output	Meaning
`yogurt_weekly_flavor_quantities.csv`	Sparse weekly flavor quantities
`yogurt_weekly_baskets.csv`	Basket totals, labels, gaps, eligibility, spell IDs
`yogurt_transition_counts.csv` / `yogurt_transition_probabilities.csv`	Eligible adjacent single-flavor transitions
`yogurt_single_flavor_spells.csv`	Consecutive single-flavor run lengths
`yogurt_dynamics.csv/.tex`	Dynamics overview
`flavor_hhi_histogram.pdf`, `flavor_hhi_cdf.pdf`	Equal-household concentration distributions
`flavor_shares.pdf`, `flavor_transitions.pdf`	Up-to-20-flavor visual summaries
`aggregated_flavors/`	Corresponding results for other/berry/plain
`yogurt_estimation_sample.parquet` under output	Full retained purchase columns with saved grouped flavor

The final export reopens the original master sample, filters years/positive prices/eligible households, drops an existing flavor column if necessary, and joins the exact grouped mapping. It refuses to overwrite the original source path. It preserves store and DMA fields not needed by demographic projection.

Synthetic regression checks can run without private data:

```
python3 -m unittest -v test_summ_stats_memory.py
```

Related summary tests live in `../est_code/test_summary_stats.py`. The memory checks compare outputs across partition counts and exercise preservation of original purchase metadata. They validate controlled examples; private-data coverage, price conventions, and join losses still require data audits.

When totals look surprising, trace the first stage where they change: source year coverage, trip matching, retail matching, price coverage, two-week selection, flavor matching, and finally estimator exclusions. Package shares, equal-household demographics, transition pairs, and spell means use different denominators. Their differences are often explained by their units rather than a calculation error.

Demand estimation: from purchase histories to a simulated likelihood

This chapter explains `est.py`, its four entry points, `price_control.py`, and `bootstrap_se.py`. It follows the current source rather than treating older README paragraphs as authoritative. For the upstream construction, see the data chapter; for the separate theoretical pricing exercise, see the supply chapter.

Contents

1. Economic question and observation units
2. Source map and execution order
3. Purchase records, weekly counts, and menus
4. History construction and a worked example
5. Utility, normalizations, and heterogeneity
6. Household likelihood and stable integration
7. Analytic score and optimization
8. Other-DMA instruments and residual controls
9. Household bootstrap
10. Commands, outputs, diagnosis, and extensions

1. What is being estimated?

The baseline question is: conditional on a household buying yogurt in an observed week, how do its flavor purchases depend on prices and its past flavor purchases? The alternatives, in fixed array order, are `other`, `berry`, and `plain`. They are groups of UPCs, not brands or individual products.

There are four distinct units. An input row is a purchase line. Prices originate as store/week/UPC quotes. The choice observation is a household-week count vector. The unit over which persistent random preferences are integrated and resampled is the household. Mixing these units up changes the estimator.

Write household as i, calendar week as t, and flavor as j. The observed count q_{ijt} counts packages, and $Q_{it} = \sum_j q_{ijt}$. A week buying two berry packages and one plain package is one row with counts $(0, 2, 1)$, not a categorical label of "berry." The model conditions on the observed number of packages. It does not explain the decision to buy three packages rather than one.

By default `capacity=0`: there is no outside alternative. Missing purchase weeks are not inserted. This is central to interpretation: category participation, total yogurt demand, and substitution into non-yogurt goods are not estimated.

2. Source map and control flow

Source	Main responsibility
`est.py`	Input preparation, histories, arrays, draws, utility, likelihood, score, optimization, output
`est_naive.py`	Calls `main` with model `naive`
`est_gamma.py`	Calls `main` with model `homogeneous`
`est_het_naive.py`	Calls `main` with model `mixed_naive`
`est_het_gamma.py`	Calls `main` with model `heterogeneous`
`price_control.py`	DMA validation, fixed-effect absorption, flavor first stages
`bootstrap_se.py`	Whole-household resampling, repeated two-stage fitting, inference
`test_est.py`	Synthetic checks of histories, preparation, likelihood, gradients, and related behavior
`test_price_control.py`	Synthetic price-control and bootstrap checks
`test_summary_stats.py`	Checks of upstream summary behavior

`main()` parses and validates arguments, calls `prepare_data`, optionally calls `add_price_controls`, saves weekly inputs, creates numerical arrays, and calls `fit_model`. The latter calls `optimize_demand`, saves point estimates and the gamma distribution, runs inference if appropriate, and writes final output.

The scripts import sibling modules directly. Run commands from this directory or invoke the script by its path. The code is not installed as a Python package. Importing `est.py` does not read the private input data.

3. Turning purchase lines into weekly choice observations

3.1 Inputs and initial checks

The default prepared input is the file `yogurt_estimation_sample.parquet` under the following directory:

```
/scratch/dtm63837/Kilts_Panel/Data/summary_outputs/
```

Its important fields are household ID, panel year, purchase date, trip ID, UPC, store ID, positive integer quantity, flavor, and merged price. With price controls, the consumer DMA is required as well. `--household-dma-column` changes its source name. `price` takes precedence over `price_rms`.

`prepare_data(args)` scans with Polars, restricts panel years, normalizes UPCs, and uses saved flavor labels if present. Only older inputs without a `flavor` column require `read_flavors`. Saved labels outside the three groups stop the run. Invalid/nonpositive prices are filtered before weekly histories are built. Quantities in the loaded demand sample must be finite, positive, and integer-valued.

UPC normalization is string based: `"000123.0"` becomes `"123"`. Avoiding a floating-point conversion protects long identifiers. `read_flavors` can read an already resolved `upc,flavor` mapping, which must have unique normalized UPCs, or classify descriptive text with plain precedence over berry terms. Its legacy recovery path supplements absent UPCs from product metadata.

The development option `--max-households` takes the smallest household IDs. It is deterministic, not a random subsample. The quote universe still comes from the full year-filtered input, so limiting demand households does not deliberately shrink their menus or the instrument universe.

3.2 Calendar time

The week ends on Saturday. Polars weekdays are Monday=1 through Sunday=7, so the current offset is $(6 - \text{weekday}) \bmod 7$. Saturday has offset zero and Sunday offset six. A Sunday purchase therefore belongs to the following Saturday, not the preceding one.

The estimator recomputes weeks from purchase dates. The checked-in upstream `panel_week.py` formula agrees with it under Polars' 1–7 weekday convention. Older notes warn of Sunday losses; that warning requires checking the historical code/data version rather than assuming the current formula is wrong. If an earlier build did use

incorrect weeks, recomputation cannot restore purchases already discarded by its inner join. The data chapter works through both formulas.

3.3 Counts and exposure stores

Package counts sum purchases across **all stores** in a household-week. The exposure store is the store at which the household bought the most packages that week; a tie goes to the lowest store code. All alternatives are priced using that exposure store's menu. Counts and prices consequently have different aggregation rules. A purchase at a second store still contributes to the count.

For prices, duplicate store/week/UPC quotes are removed before averaging. A store/week/flavor price is the arithmetic mean of its observed UPC quotes, with one weight per UPC quote. It is not expenditure per package, a quantity-weighted average, or an average of the household's purchases. No `prmult` adjustment is made. Conflicting price or flavor values for the same quote key stop preparation.

Because quotes are recovered from merged purchase records, a price absent here does not establish that the store did not offer the product. The menu is a maintained measurement rule based on available records.

3.4 Availability and the order of exclusions

A finite positive flavor price makes that alternative available. An unpriced flavor with zero purchases is simply excluded from that week's denominator. An unpriced flavor with positive purchases invalidates the week. With controls, a missing other-DMA instrument also invalidates the likelihood observation.

The order is deliberate:

1. Form weekly counts and attach menus/instruments.
2. Build histories from all these observed positive-price purchase lines.
3. Exclude weekly likelihood observations with incompatible menus or instruments.
4. Keep households with at least two remaining usable weeks.

A week excluded at step 3 can affect later histories. Rebuilding histories after that exclusion would erase a purchase that the researcher actually observed. However, purchase lines removed before weekly aggregation never enter history.

4. The history index, derived and worked through

`add_stocks` works household by household in calendar order. Define within-week package shares

$$x_{ijt} = q_{ijt}/Q_{it}.$$

The own-flavor stock is

$$S_{ijt} = \sum_{s<t,\ s \text{ observed}} x_{ijs}.$$

Every observed purchase week contributes total stock one across flavors, regardless of how many packages it contains. Two berry packages and twenty berry packages each add one to berry stock if berry is the only flavor purchased. There is no depreciation, finite window, or division by panel length.

The recent-other-flavor term is

$$N_{ijt} = \sum_{k \neq j} x_{ik,t-1}, \qquad \Xi_{ijt} = (1-\lambda)N_{ijt} - \lambda S_{ijt}.$$

Here $t-1$ means the immediately preceding calendar week. After a gap, $N = 0$ for every flavor. The cumulative S persists across gaps and years. `--habit-weight` sets λ, default 0.7. Both terms start at zero on a household's first observed week. Current purchases update the stock only after the current index is recorded.

Consider this history in the code's flavor order:

Week	Current counts	Pre-choice stock S	Recent-other N	Xi at lambda=.7
Jan 2	(0,2,0)	(0,0,0)	(0,0,0)	(0,0,0)
Jan 9	(1,1,0)	(0,1,0)	(1,0,1)	(.3,-.7,.3)
Jan 23	(0,0,3)	(.5,1.5,0)	(0,0,0)	(-.35,-1.05,0)

Jan 9 adds half a stock unit to other and half to berry. Jan 23 has no recent term because Jan 16 has no observed purchase. Its current plain purchase affects future stock, not its own current Xi.

Positive gamma lowers utility for a flavor with high own stock and raises it for alternatives different from recent purchases. Negative gamma reverses both effects and represents habit/familiarity under this specification. Gamma is allowed to be negative. Novelty does not mean "never tried": it refers to a flavor different from recent purchases.

Only one gamma scales the fixed combination. The code does not independently estimate habit and novelty effects. Unobserved pre-panel history is set to zero; missing purchases also mean missing history. Since stock grows with observed panel length, panel-length and initial-condition sensitivity matter.

5. Utility and the four specifications

For an available inside flavor,

$$V_{ijt,r} = \beta_j + \gamma_r \Xi_{ijt} + \alpha_r p_{ijt} + \sigma \hat{\varepsilon}_{ijt}.$$

The final term exists only with price controls. The normalization is $\beta_{other} = 0$, with common coefficients for berry and plain. Conditional choice only identifies relative utilities: adding the same constant to every inside alternative cancels from probabilities.

Model argument	Price coefficient	History coefficient
`naive`	Common negative alpha	Zero
`homogeneous`	Common negative alpha	Common unrestricted gamma
`mixed_naive`	Random negative alpha	Zero
`heterogeneous`	Random negative alpha	Random normal gamma

The heterogeneous specification uses independent standard normal coordinates:

$$\alpha_r = -\exp(\mu_a + \tau_a z_{r,a}), \qquad \gamma_r = \mu_\gamma + \tau_\gamma z_{r,\gamma}.$$

Alpha is always negative. Gamma has support on both signs. Independence is a restriction: correlated price sensitivity and history preferences are not estimated. Beta never varies across draws or households.

`mu_log_minus_alpha` is μ_a, and `sd_log_minus_alpha` is τ_a, not the standard deviation of alpha in dollars. The implied moments are

$$E[\alpha] = -e^{\mu_a + \tau_a^2/2}, \qquad \mathrm{Var}(\alpha) = (e^{\tau_a^2} - 1)e^{2\mu_a + \tau_a^2}.$$

For $\tau_\gamma > 0$, the habit share is $P(\gamma < 0) = \Phi(-\mu_\gamma/\tau_\gamma)$. A negative population mean does not imply every household has negative gamma. These are population type distributions; the code does not output household posterior coefficient estimates.

The optional constructed outside count

For positive capacity K, the code sets $q_{i0t} = \max(K - Q_{it}, 0)$, adds outside utility zero, and estimates a common inside constant. With `--capacity 7`, a three-package week gets outside count four; a nine-package week gets zero. Missing weeks remain absent.

This is a constructed outcome, not measured nonpurchase. Total modeled count is $\max(K, Q_{it})$, and package counts proxy occasions. Changing K changes the outcome and the likelihood, so its NLL cannot be compared directly to capacity zero as if only a parameter restriction changed.

6. The likelihood: why integration comes after summing the history

For available set A_{it}, the conditional logit probability is

$$P_{ijt,r} = \frac{e^{V_{ijt,r}}}{\sum_{k \in A_{it}} e^{V_{ikt,r}}}.$$

Include outside in this denominator only when enabled. Conditional on a type r, the household's count-logit log likelihood is

$$\ell_{ir} = \sum_t \sum_j q_{ijt} \log P_{ijt,r}.$$

The multinomial coefficient $Q!/\prod_j q_j!$ is omitted because it does not depend on parameters. Thus reported NLL is suitable for optimizing the same outcomes, but is not the fully normalized multinomial log likelihood.

One household has the same latent type in all its weeks. The simulated likelihood is

$$\widehat{L}_i = \frac{1}{R} \sum_{r=1}^{R} e^{\ell_{ir}}, \qquad \log \widehat{L}_i = \mathrm{LSE}_r(\ell_{ir}) - \log R.$$

The sum over weeks is **inside** the integration. Integrating probabilities separately each week would redraw preferences every week and define another model. A household with many repeated berry choices can make a persistent berry- favoring type much more plausible than a sequence of independently redrawn types.

`make_arrays` produces counts of shape W-by-J, prices/Xi/residuals of shape W-by-3, a W-by-3 availability mask, and a length-W dense household index. `np.bincount` aggregates weekly log likelihood contributions to households. Unavailable utilities become minus infinity. Their zero purchase counts are handled explicitly so the code does not multiply zero by minus infinity.

`logsumexp` subtracts the largest exponent internally, avoiding overflow in both choice denominators and household integration. `household_loglike` processes one draw at a time rather than allocating a W-by-R-by-J utility tensor.

`make_draws` requires an even R of at least two. It generates R/2 independent two-dimensional standard normal draws and appends their negatives. These antithetic draws are fixed throughout a fit and shared as quadrature nodes across households. They approximate the population integral; they are not estimated household identities. Nonmixed models use a single zero draw.

7. The analytic gradient and numerical optimization

`demand_objective_gradient` returns mean household NLL and its analytic gradient. For one type draw, the weekly utility score for alternative j is

$$e_{ijt,r} = q_{ijt} - Q_{it}^{model} P_{ijt,r}.$$

For a parameter θ_k, multiply this residual by $\partial V_j/\partial\theta_k$, sum alternatives and weeks, then integrate with the household's posterior draw weights:

$$w_{ir} = \frac{e^{\ell_{ir}}}{\sum_a e^{\ell_{ia}}}, \qquad \frac{\partial \log \widehat{L}_i}{\partial \theta_k} = \sum_r w_{ir} \frac{\partial \ell_{ir}}{\partial \theta_k}.$$

This is the score of the finite-draw simulated likelihood, not a finite-difference approximation. The chain rule gives price derivatives $\alpha_r p_j$ for μ_a, $z_{r,a}\alpha_r p_j$ for τ_a, Xi for mean gamma, and $z_{r,\gamma}\Xi$ for gamma SD. Berry/plain intercept scores select their respective residuals. The residual-control coefficient uses the first-stage residual.

The optimizer minimizes average NLL for scale stability. Printed/checkpointed NLL multiplies it by the household count. `L-BFGS-B` uses the supplied gradient, `ftol=1e-10`, `gtol=1e-6`, and default maximum 600 iterations.

Bounds are implementation restrictions:

Parameter	Bounds	Initial value
Any `sd_` parameter	[0,3]	.3
`mu_log_minus_alpha`	[-6,3]	0
All remaining parameters	[-20,20]	0

Starting SDs away from zero avoids a symmetric stationary point. The first start uses this vector; later starts add seeded normal noise with SD .5 and clip to bounds. Default main starts are three. This is local multistart optimization, not a proof of a global optimum. Importantly, selection keeps the smallest objective even if that candidate did not converge; final convergence must be read from `fit.json`. Boundary distance below `1e-4` produces `at_bound=True`.

More draws improve integration accuracy, more starts probe local solutions, and more iterations allow longer searches. They address different problems. More allocated CPUs alone do not parallelize the draw or bootstrap loops.

8. The price control, step by step

8.1 The instrument actually used

For week t, let T_t, C_t be the sum and count of valid unique quotes over all consumer DMAs and flavors, and T_{dt}, C_{dt} those in consumer DMA d. Then

$$Z_{dt} = \frac{T_t - T_{dt}}{C_t - C_{dt}}.$$

Each quote is deduplicated within DMA by store/week/UPC/flavor/price. The same physical quote can appear in multiple consumer DMAs if shoppers from different DMAs visited that store; this is a property of the merged input. DMA is the consumer's `dma_cd`, not automatically the retail store's DMA.

Z pools all flavors, weights quotes equally, and is copied into `iv_other`, `iv_berry`, and `iv_plain`. It is not a mean of DMA means or a flavor-specific UPC-matched instrument. A denominator of zero yields missing Z and excludes the observation after history construction.

`price_control.other_dma_instruments` implements a different, UPC-matched retail-DMA construction retained in the module. It is **not called by the current `prepare_data` path**. Its presence does not describe the active instrument.

8.2 First-stage regressions

For each flavor j, `add_price_controls` estimates on usable observations with that flavor available:

$$p_{ijt} = \pi_j Z_{dt} + b_j' \Xi_{it} + \eta_{jt} + \kappa_{jd} + \varepsilon_{ijt}.$$

The three Xi columns all enter each equation. There are flavor-specific week and DMA fixed effects. Every household-week gets equal weight; a shared menu price can repeat across many households. Availability creates different samples for the three equations.

`absorb_effects` repeatedly subtracts group means for week and DMA from the matrix containing y, Z, and Xi. In an unbalanced panel, one pass generally reintroduces means for the other effect. The code cycles until the largest change is below `1e-10`, with at most 2,000 cycles.

After absorption, the Frisch–Waugh–Lovell calculation projects Z and y onto Xi, then estimates $\hat{\pi} = (Z_\perp' y_\perp)/(Z_\perp' Z_\perp)$. Least squares tolerates redundant Xi columns. An excluded instrument with no remaining variation stops the fit. An essentially zero residual also stops it.

The resulting residual is in dollars and enters utility with one common unrestricted `price_control` coefficient. An unavailable flavor gets placeholder residual zero and is masked out anyway. No outside residual is constructed.

8.3 Diagnostics and identification

Household-cluster scores for the IV slope are $g_i = \sum_{t \in i} Z_{\perp,it} \hat{\varepsilon}_{it}$. The reported variance uses $\sum_i g_i^2/(Z_\perp' Z_\perp)^2$, multiplied by H/(H-1) when H>1. The squared clustered t statistic supplies `iv_cluster_F`; `partial_R2` compares the unrestricted residual sum of squares to y after Xi partialling.

These describe relevance conditional on the included controls. They do not establish exclusion. Common demand shocks, national promotions, and cross-market pricing policies can connect other-DMA prices to local unobserved demand. The linear residual control also imposes a functional-form restriction. Disabling controls with `--no-price-control` removes the IV requirement, first stages, and utility residual term, potentially changing the sample as well as the model.

9. Household bootstrap and its scope

`resample_households` samples H household histories with replacement. Selecting the same original household twice creates two new IDs, preserving two independent bootstrap contributions. All retained observations and precomputed Xi travel with the history. Xi is not recomputed from the reduced likelihood sample.

Each replication then refits all enabled price equations and reoptimizes demand. The main estimate supplies the initial vector, with default two local starts. The integration seed and nodes remain fixed, as do constructed instruments and the underlying retail quote universe. Thus this is two-stage household sampling inference conditional on the observed prices/instruments and integration scheme. It does not estimate uncertainty from common DMA/time shocks or simulated integration.

Only converged, finite bootstrap fits enter inference. Expected identification or linear-algebra failures are recorded. After every replication the complete record list is checkpointed to `bootstrap_draws.csv`.

Let B be requested repetitions and V successful fits. Reporting requires $V \geq \max(20, \lceil .8B \rceil)$. For B=49 this means at least 40 successful fits. Reported SEs are sample standard deviations; limits are empirical 2.5 and 97.5 percentiles. Covariance uses the same accepted vectors. Missing SEs mean disabled, skipped, or insufficient inference, not zero uncertainty.

Main nonconvergence skips the bootstrap entirely. Boundary SDs and selected-out failed repetitions complicate interpretation even if the numerical threshold passes. A bootstrap cannot repair misspecified histories or an invalid instrument.

10. Running and reading the results

From `final_draft/est_code`, use the existing environment with dependencies from `requirements.txt`. For a development run:

```
python3 est.py --sample /path/to/summary_outputs/yogurt_estimation_sample.parquet \
  --max-households 100 --draws 32 --starts 1 --se-method none \
  --output /path/to/results/development
```

For a larger run, choose a separate output directory:

```
python3 est_het_gamma.py --sample /path/to/summary_outputs/yogurt_estimation_sample.parquet \
  --draws 128 --starts 5 --bootstrap-reps 49 \
  --output /path/to/results/heterogeneous
```

Forty-nine is the current bootstrap default, not an assertion that it suffices for final inference. Increasing to 499 improves tail resolution if computation and fit stability permit. `--seed` defaults to 306, years to 2006–2015, and `--habit-weight` to .7. Run `python3 est.py --help` for the complete interface.

Output	How to use it
`weekly_inputs.csv`	Audit actual counts, prices, availability, histories, instruments, residuals
`first_stage.csv`	Flavor-specific IV slope, clustered SE/F, partial R^2, residual SD, sample sizes
`starts_checkpoint.json`	Completed starts, parameter vectors, NLL, convergence, iteration counts
`point_estimates.csv`	Main fit saved before the potentially long bootstrap
`gamma_distribution.pdf`	Fitted population gamma distribution, with provisional-fit marking
`bootstrap_draws.csv`	Successes, failures, messages, and coefficients after every replication
`covariance.csv`	Accepted bootstrap covariance, when reporting criteria pass
`estimates.csv`	Estimates, boundary flags, SEs, percentile limits
`fit.json`	Final convergence, optimizer message, settings, NLL, starts, inference status

Repeated use of an output directory can leave old artifacts from another model or from previously enabled inference. Prefer a fresh directory per specification. Checkpoint files record progress; the code does not implement automatic resume.

Inspect in dependency order. First verify sample and menu construction. Then check instrument variation and residual magnitudes. Then compare starts and boundary flags. Next increase draws and change seeds to assess integration sensitivity. Only then interpret bootstrap uncertainty. Compare NLL across models only with identical households, weeks, availability, capacity, and counts.

For synthetic checks without Nielsen data:

```
python3 -m unittest -v test_est.py test_price_control.py test_summary_stats.py
```

Modifying the estimator carefully

Adding a coefficient involves `parameter_names`, utility construction in both likelihood paths, the analytic score, and appropriate bounds/initialization. Changing only `household_loglike` leaves optimization on a different model, because it uses `demand_objective_gradient`. Changing the flavor set also touches fixed array order, reference labels, price controls, output columns, and tests. Changing histories must preserve the timing convention and the distinction between observation exclusions and information available for later states.

Finally, these estimates do not feed automatically into supply. Their alpha is attached to merged package prices, gamma multiplies cumulative Xi, and beta is relative to other flavor. Supply's fixed-type binary switching parameters need a separate calibration or an explicitly redesigned empirical model.

Dynamic pricing: a two-period symmetric-switching model

This chapter derives the current pricing model and connects it to `model/`, `pricing/`, `comparative_statics/`, numerical evaluation, verification, and exports. `MODEL_CONVENTIONS.md` describes the current conventions. The estimation chapter explains a related but distinct empirical model; this code does not automatically consume its estimates.

Contents

1. Economic setting and state timing
2. File map and source of truth
3. Raw utilities and outside normalization
4. Terminal pricing and value functions
5. First-period pricing by backward induction
6. Numerical solution and uniqueness
7. Comparative statics from implicit differentiation
8. Gamma effects, curvature, and profit
9. Price ratios and conditional interpretations
10. API, figures, tests, and extensions

1. The economic problem

A firm sells one inside product j against an outside alternative 0 over two periods. Each consumer type is fixed and observed for this pricing exercise. The firm has constant marginal cost c and discounts second-period profit by delta. Demand is binary logit. The state records the consumer's last choice, which changes the attractiveness of switching versus staying.

The timing is:

1. Start with observed history $x_0 \in \{j, 0\}$.
2. Set first-period price $p_1(x_0)$; choice x_1 occurs.
3. Observe x_1, then set $p_2(x_1)$.
4. Terminal choice occurs and the horizon ends.

Both second-period branches matter when choosing the first-period price. A consumer who chooses outside in period 1 is still present in period 2. The firm does not receive continuation value only after a sale.

The maintained parameter domain is alpha<0, fixed finite beta/gamma/c, and 0<delta<1. The numerical implementation also permits delta=0 as a static counterfactual. Prices range over all real numbers. A first-period subsidy can be optimal, and the solver deliberately does not impose price at least zero or at least cost. Shares are strictly interior at finite utility indices.

This is a finite-horizon monopoly exercise conditional on a type and initial history. It does not solve rival-firm equilibrium, a household type mixture, an infinite-horizon Bellman equation, or an estimated multi-flavor model.

2. Architecture and source of truth

File/module	Role
`model/symbols.py`	Shared SymPy symbols, shares, branch prices, values, signed histories
`model/assumptions.py`	Human-readable maintained assumptions
`model/utility.py`	State mapping, raw utilities, normalized utilities
`model/shares.py`	Logit expressions and direct share derivatives
`model/profit.py`	Current profit, both continuation branches, total profit
`pricing/period2_foc.py`, `period2_price.py`	Terminal FOC and implicit markup equation
`pricing/period1_foc.py`, `period1_price.py`	Dynamic FOC and continuation adjustment
`comparative_statics/canonical.py`	Production derivative definitions
Other `comparative_statics/*.py`	Named derivative interfaces built from the common model
`model/numerical.py`	Backward induction, root solving, lambdified derivative evaluation
`structural_switching_reference.py`	Independent verification oracle retained from the handoff
`checks/test_symmetric.py`	Current integrated symbolic/numerical verification suite
`make_comparative_static_figures.py`	Benchmark grid, six figure pairs, solution CSV
`make_latex_document.py`	Current mathematical/figure document generation
`export/`	Symbol formatting and LaTeX helpers
`run_all.py`	Current reproducibility entry point

The conceptual dependency order is utilities → shares/profit → FOCs → implicit derivatives → numerical evaluation → figures/document. The numerical evaluator lambdifies expressions from `canonical.py`; the plotting script does not maintain its own hand-copied derivative formulas.

`assumptions.py` stores descriptions; it does not make SymPy infer every inequality. Many shared parameters are declared real, and share symbols positive. Economic domain restrictions must still be supplied when interpreting signs.

Historical notes `DERIVATIVE_CHANGES.md` and `RUN_ALL_DIAGNOSIS.md`, and older figure/CSV files, can concern superseded models. The old entry-only formulation, 0/1 state coefficients, and global ratio-sign claims do not describe current production behavior. The old `z1,z2` numeric interface raises a migration error. Symbolic aliases z1/z2 now mean signed q/q2, not old binary indicators.

3. Utilities: why the history coefficient is signed

Define a state sign

$$q(j) = -1, \qquad q(0) = +1.$$

`signed_state` accepts exactly the strings `"j"` and `"0"`. The latter is a string state name, not a numeric zero passed as a history coefficient.

`raw_utilities(price,q)` returns

$$U_j^{raw} = \beta + \alpha p + \gamma \frac{1+q}{2}, \qquad U_0^{raw} = \gamma \frac{1-q}{2}.$$

Thus a previously outside consumer gets gamma on the inside alternative, while a previously inside consumer gets gamma on the outside alternative. Positive gamma rewards switching in either direction. Negative gamma rewards staying.

Subtracting outside utility from both alternatives preserves choice probabilities:

$$U_j = \beta + \alpha p + q\gamma, \qquad U_0 = 0.$$

This normalization is not the assumption that the outside alternative has no history effect. That effect has been subtracted from the inside utility. Missing it would remove switching incentives after inside consumption.

The inside share is

$$s(p,q) = \frac{1}{1+\exp[-(\beta+\alpha p+q\gamma)]}, \qquad s_p = \alpha s(1-s).$$

At a fixed price, $s_\gamma = qs(1-s)$. This is a direct derivative; after reoptimization the price also changes. `model/shares.py` keeps these partial derivatives separate from total derivatives in `canonical.py`.

4. Solving the terminal period

For a realized second-period history q2, the firm solves

$$V_2(q_2) = \max_{p_2}(p_2 - c)s(p_2, q_2).$$

Differentiate:

$$0 = s_2 + (p_2 - c)\alpha s_2(1 - s_2).$$

Since $s_2 > 0$, divide through to obtain

$$1 + \alpha(1-s_2)(p_2-c) = 0, \qquad p_2^* = c - \frac{1}{\alpha(1-s_2)}.$$

This is an **implicit** price equation: s2 depends on the unknown optimal price. The code's symbolic `p2_star` is a markup representation, not an explicit solution that can be evaluated with an arbitrary fixed share.

Solve both histories:

$$p_{2j} = p_2^*(j), \quad p_{20} = p_2^*(0), \quad u = s(p_{2j}, -1), \quad v = s(p_{20}, +1),$$

$$V_j = (p_{2j} - c)u, \qquad V_0 = (p_{20} - c)v, \qquad \Delta V = V_j - V_0.$$

Terminal prices exceed cost since alpha<0. At gamma=0 the two branches coincide. For positive gamma the outside-history branch has higher utility intercept, so it has a higher optimized value; $\Delta V < 0$. For negative gamma, inside-history continuation is more valuable and $\Delta V > 0$.

5. First-period profit and the effective-cost interpretation

Let $s = s(p_1, q)$ for the observed initial history. Full expected profit is

$$\Pi(p_1) = s(p_1 - c) + \delta[sV_j + (1-s)V_0].$$

Conditional on each realized branch, Vj and V0 do not depend on p1: second-period price is reset after observing the state, and there are no other carried state variables. They do depend on structural parameters such as gamma. Distinguishing these statements is essential when differentiating profit.

Rewrite the objective as

$$\Pi(p_1) = \delta V_0 + s[p_1 - c + \delta \Delta V].$$

The firm chooses p1 as if it were a static logit monopolist with effective cost $c_{eff} = c - \delta \Delta V$. Its FOC is

$$0 = s + \alpha s(1 - s)[p_1 - c + \delta \Delta V],$$

and the corresponding price equation is

$$p_1^* = c - \frac{1}{\alpha(1 - s)} - \delta \Delta V.$$

When acquisition creates a more valuable continuation state, the firm lowers current effective cost and may subsidize the initial purchase. When a current purchase sends the consumer to a less valuable state, the dynamic adjustment raises price. The firm's gain from inducing a sale is a **difference** in future values, because future profit also exists after outside choice.

The evaluator reports `static_component` as $c - 1/[\alpha(1 - s)]$, using the **dynamic** optimal share. It reports `continuation_adjustment` as $-\delta \Delta V$. Their sum is p1. The separately optimized `static_counterfactual` sets effective cost to c and reoptimizes its share. It generally differs from `static_component`.

At gamma=0, $\Delta V = 0$, so dynamic p1 equals the true static price even with positive delta. Total profit still includes a second-period value.

6. The numerical root solver and why its solution is unique

`optimal_price(intercept, alpha, effective_cost)` solves the same scalar problem for each terminal branch and for period 1. Let A denote the utility intercept and k effective cost. The FOC can be written

$$f(p) = \alpha(p - k) + 1 + \exp(A + \alpha p) = 0.$$

At p=k, f is positive. As p tends to positive infinity, f tends to minus infinity. Moreover,

$$f'(p) = \alpha[1 + \exp(A + \alpha p)] < 0.$$

There is exactly one root above k. The derivative of the profit objective has the sign of f because its remaining factors are positive; profit rises before the root and falls after it. This establishes the global optimum of this scalar unconstrained problem, not merely a stationary point.

The implementation starts a bracket at k, sets initial width `max(2,2/abs(alpha))`, and repeatedly doubles it until the upper endpoint has negative f. `brentq` then finds the root with `xtol=1e-12`. Exponents are clipped to [-745,700] for numerical range, and share computation uses stable `expit`. Clipping is a floating-point safeguard, so extreme calibrations beyond the benchmark warrant additional numerical scrutiny.

`evaluate` validates finite parameters and alpha<0, 0<=delta<1, then:

1. Solves p2j and p20 with effective cost c.
2. Computes u, v, Vj, V0, and their difference.
3. Solves p1 using effective cost c-delta*DeltaV.
4. Computes first-period share and complete two-branch profit.
5. Evaluates lambdified symbolic derivatives at that solution.
6. Reports decomposition terms, counterfactual price, ratio derivatives, and FOC residuals.

There is no need for a simultaneous three-price optimizer: backward induction uses terminal separability. FOC residuals directly check the original profit equations, not only the transformed scalar root.

7. General comparative statics: the implicit-function calculation

`canonical.py` centralizes derivatives for any symbolic parameter theta. Let the direct utility-index derivative at fixed price be

$$D_{2\theta} = \beta_\theta + p_2\alpha_\theta + q_2\gamma_\theta.$$

Differentiating the terminal price equation while allowing its share to respond gives the implemented expression

$$\frac{dp_2^*}{d\theta} = (1 - s_2)c_\theta + \frac{\alpha_\theta}{\alpha^2} - \frac{s_2 D_{2\theta}}{\alpha}.$$

`price2_derivative(theta, share, history, price)` supplies this formula for an arbitrary branch. For beta it is $-s_2/\alpha > 0$; for gamma it is $-q_2 s_2/\alpha$; for delta it is zero. There is no continuation after period 2, so the terminal price cannot depend on discounting.

The envelope theorem eliminates the derivative of the optimized price from the branch-value derivative:

$$\frac{dV_2}{d\theta} = -\frac{s_2 D_{2\theta}}{\alpha} - s_2 c_\theta.$$

This follows by differentiating profit at fixed p2 and using the terminal FOC. `value2_derivative` implements it. It avoids needlessly expanding every price response inside continuation values.

For period 1, with $D_{1\theta} = \beta_\theta + p_1\alpha_\theta + q\gamma_\theta$,

$$\frac{dp_1^*}{d\theta} = (1 - s)c_\theta + \frac{\alpha_\theta}{\alpha^2} - \frac{sD_{1\theta}}{\alpha} - (1 - s)[\delta_\theta \Delta V + \delta \Delta V_\theta].$$

The factors (1-s) reflect the feedback of price into its own demand. Holding the optimal share fixed while differentiating the markup equation would miss that feedback. `price1_derivative` is the production implementation.

For second and mixed derivatives, `total_derivative(expr,theta)` adds explicit parameter differentiation and the chain-rule effects through p1, p2j, p20, s, u, v, Vj, and V0. A plain `sp.diff(expr,gamma)` treats independent share symbols as fixed and therefore is insufficient. `terminal_total_derivative` does the smaller single-branch calculation. Mixed beta-gamma modules use these same helpers rather than a separate model.

8. Gamma comparative statics and their interpretation

8.1 Terminal responses

Substituting signed histories into the preceding formulas gives

$$p'_{2j} = u/\alpha < 0, \qquad p'_{20} = -v/\alpha > 0,$$

$$V'_j = u/\alpha < 0, \qquad V'_0 = -v/\alpha > 0, \qquad \Delta V' = (u + v)/\alpha < 0.$$

Primes in this section mean total derivatives with respect to gamma. More switching preference makes the previously inside branch less attractive and the previously outside branch more attractive. Optimal prices respond in the same directions as those branch demand intercepts.

The reoptimized share derivatives are

$$u' = -u(1 - u)^2, \qquad v' = v(1 - v)^2.$$

They differ from direct share responses -u(1-u) and v(1-v) because prices also move.

8.2 First-period response and conditional signs

The current source simplifies to

$$p_1' = -\frac{qs + \delta(1-s)(u+v)}{\alpha}.$$

For an initially outside consumer q=+1, the numerator is positive, so p1'>0. For an initially inside consumer q=-1, the direct history effect opposes the continuation effect. The price derivative is positive precisely when

$$\delta(1-s)(u+v) > s,$$

zero at equality and negative otherwise. A common global sign across initial histories is therefore unwarranted.

The total first-period share derivative is

$$s' = s(1-s)^2[q - \delta(u+v)].$$

For q=-1 it is negative. For q=+1 its sign depends on the size of the dynamic response. A positive direct demand effect can be offset by optimal repricing.

8.3 Curvature

`d2p1_dgamma2` is a total second derivative. One useful checkable representation is

$$p_1'' = -\frac{1}{\alpha}\left\{s(1-s)^2[q-\delta(u+v)]^2 + \delta(1-s)[-u(1-u)^2 + v(1-v)^2]\right\}.$$

The squared term is nonnegative; the difference of terminal share responses can have either sign. Hence curvature is not established by inspecting only the square. The tests compare this expression both to derivatives of p1' and to second finite differences of fully reoptimized prices.

8.4 Optimized profit

`profit_derivative` applies the first-period envelope and terminal envelopes. It includes both branches and their state probabilities. For gamma:

$$\frac{d\Pi^*}{d\gamma} = \frac{-qs + \delta[su - (1-s)v]}{\alpha}.$$

The absence of explicit p1' does not mean price is fixed in the optimized profit function. The envelope theorem cancels its first-order contribution at the optimum. This formula is not the derivative of only current revenue. Its sign can depend on initial history and calibration; current figures show the conditional curves rather than asserting one universal effect.

9. Three price ratios that answer different questions

The code studies state-matched ratios

$$R_j = p_1(j)/p_{2j}, \qquad R_0 = p_1(0)/p_{20},$$

and an acquisition-conditioned induce/harvest ratio

$$R_{0\to j} = p_1(0)/p_{2j}.$$

The last ratio compares the introductory price for an initially outside consumer to the terminal price conditional on that consumer buying inside in period 1. It is not an unconditional expected second-period price. The matched outside ratio conditions on the outside continuation state instead.

All ratio derivatives use the quotient rule:

$$R' = \frac{p_2 p_1' - p_1 p_2'}{p_2^2}.$$

Increasing both prices does not determine the ratio's direction; their relative changes matter. Negative introductory prices also undermine casual percentage- growth interpretations. If c is sufficiently negative, a terminal price can approach zero and the ratio becomes ill-conditioned or undefined. The benchmark c=0 has positive terminal prices; the generic API does not impose c>=0.

`matched_j`, `matched_0`, and `induce_harvest` are derivative fields, not ratio levels. `evaluate` sets state-inapplicable fields to NaN and supplies convenient `d_matched_ratio` and `d_induce_ratio` fields. Do not interpret NaN there as a solver failure. Ratio levels can be calculated from the price fields when the denominator is safely nonzero.

10. Using the API and reproducing outputs

From `final_draft/supply_code`:

```
from model.numerical import evaluate, solve

r = evaluate(gamma=1.0, initial_state="0", beta=2.0,
             alpha=-1.0, delta=.92, c=0.0)
print(r["p1"], r["p2j"], r["p20"], r["profit"])
print(r["dp1"], r["dprofit"], r["d_matched_ratio"])
prices_and_shares = solve(gamma=1.0, initial_state="j")
```

`evaluate` returns a dictionary. `solve` returns the ordered tuple `(p1,p2j,p20,s,u,v,profit)`. The richer dictionary is safer for analysis because field names preserve the branch meanings.

Dictionary field	Meaning
`gamma`, `initial_state`	Evaluation point and initial history
`p1`, `p2j`, `p20`	Optimal first price and both terminal prices
`s`, `u`, `v`	First-period share and two terminal inside shares
`V2j`, `V20`, `DeltaV`	Optimized branch values and difference
`profit`	Complete discounted two-period profit
`static_component`, `continuation_adjustment`	Algebraic decomposition at the dynamic share
`static_counterfactual`	Independently optimized static price
`dp1`, `d2p1`, `dprofit`	First/second gamma price derivative and profit derivative
`dp2j`, `dp20`	Terminal gamma price derivatives
`d_matched_ratio`, `d_induce_ratio`	Applicable ratio derivatives
`foc1`, `foc2j`, `foc20`	Residuals of original pricing FOCs

The reproducibility command is

```
python3 run_all.py
```

It sequentially runs the symmetric-model unittest suite, makes figures, and generates the LaTeX document with `check=True` subprocess calls. A failed step stops later steps. To compile the supply mathematical document separately:

```
latexmk -cd -pdf -interaction=nonstopmode -halt-on-error \
  writeup_math_graphs/comparative_statics_figures.tex
```

These commands regenerate research artifacts. The tutorial PDFs described in the overview are separate documentation artifacts.

Benchmark and figures

The benchmark is beta=2, alpha=-1, delta=.92, c=0. A grid of 601 gamma values from -3 to 3 is solved for each of two initial histories, giving 1,202 rows in `writeup_math_graphs/figures/solutions.csv`. Each figure is exported as PNG and PDF:

Stem	Plotted quantity
`matched_price_ratio_derivative`	Both state-matched ratio derivatives
`induce_harvest_price_ratio_derivative`	Acquisition-conditioned ratio derivative
`profit_by_gamma`	Optimized profit by initial history
`gamma_profit_by_gamma`	Optimized profit derivative
`optimal_prices_by_gamma`	First prices by initial state and both terminal prices
`gamma_p1_by_gamma`	First-price gamma derivatives

The terminal prices do not depend on x0 once x1 is realized, so the price-level figure can take terminal curves from either initial-state collection. Grid generation asserts maximum absolute FOC residual at most `1e-9`. Old files with other stems can remain in the directory and are not evidence that those old plots belong to the current model.

What verification establishes

`checks/test_symmetric.py` checks raw utility normalization, FOCs differentiated from literal profit, branch-probability derivatives, envelope identities, implicit-function identities, and gamma curvature. It compares the numerical implementation to the independent reference across the full benchmark grid.

At several other calibrations and gamma values, it compares analytic derivatives to centered finite differences of reoptimized solutions. An independent scalar profit optimizer checks p1 against direct maximization of complete two-branch profit. It also checks alpha/beta/delta/c derivatives, a mixed beta-gamma derivative, static limits, terminal-state symmetry, and negative introductory prices under strong habit.

These layers address different errors. Algebraic identities catch missing terms; finite differences catch incorrect total differentiation or argument mapping; the independent optimizer catches a solver implementing the wrong objective; static/symmetry tests catch state-convention regressions. Agreement with a reference alone would not establish the economic validity of assumptions.

For checks without regenerating figures:

```
python3 -m unittest -v checks.test_symmetric
```

Extending the model

If a parameter changes utility, update raw utilities and normalized shares, derive the profit FOCs, then propagate the change through canonical derivatives and numerical argument mapping. Generic derivative helpers assume the present linear index and transition structure; they are not automatically correct for a new state transition, nonlinear price term, or endogenous cost.

Adding a price floor requires a constrained solution and piecewise comparative statics: the current interior formulas cannot simply be evaluated at a clipped price. Adding persistent unobserved types requires a policy specifying whether the firm observes types or updates beliefs, because type-specific optimal prices cannot generally be averaged to obtain a pooled optimum. Extending to many periods requires a richer continuation recursion.

Finally, using estimated yogurt parameters requires an explicit mapping of alternatives, price units, normalizations, history units, and information. The current benchmark is a calibrated theoretical exercise with a binary last-choice state; it is not a direct supply-side estimate from the three-flavor demand output.